



INFORME TECNICO

Auditoria del Sistema ObrasJM

Evaluación integral del sistema de control y seguimiento del
Porcentaje de Avance Físico (PAF) del Hospital de Niños
J.M. de los Ríos

Proyecto:	ObrasJM - Control PAF
Organización:	Hospital de Niños J.M. de los Ríos
Fecha:	29 de May de 2026
Version:	1.0
Autor:	Z.ai - IT Development

Tabla de Contenidos

1. Resumen Ejecutivo	3
2. Arquitectura del Sistema	3
2.1 Vision General	3
2.2 Flujo de Datos	4
2.3 Single Page Architecture	4
3. Stack Tecnologico	4
4. Esquema de Base de Datos	5
4.1 Tablas Principales	5
4.2 Tipos ENUM	5
4.3 Vistas Calculadas	6
4.4 Triggers y Funciones	6
5. Autenticacion y Autorizacion	6
5.1 Flujo de Autenticacion	6
5.2 Sistema de Roles	7
5.3 Politicas RLS (Row Level Security)	7
5.4 Gestion de Contraseñas	8
6. Endpoints API	8
6.1 Catalogo de Rutas	8
6.2 Patrones de Seguridad API	9
7. Componentes Frontend	9
7.1 Estructura de Componentes	9
7.2 Patrones de Diseno UI	10
7.3 Dashboard y Visualizacion	10
8. Evaluacion de Seguridad	10
8.1 Fortalezas	10
8.2 Areas de Mejora	10

9. Deuda Tecnica y Oportunidades de Mejora	11
9.1 Deuda Tecnica Identificada	11
9.2 Mejoras Funcionales Sugeridas	12
10. Infraestructura y Despliegue	12
10.1 Configuracion de Despliegue	12
10.2 Variables de Entorno	12
11. Migraciones y Evolucion del Schema	13
12. Conclusiones y Recomendaciones	13
12.1 Estado Actual	13
12.2 Recomendaciones Prioritarias	14
12.3 Metricas deCodigo	14

1. Resumen Ejecutivo

El presente informe documenta los resultados de la auditoria tecnica realizada al sistema **ObrasJM**, plataforma de control y seguimiento del Porcentaje de Avance Fisico (PAF) de las obras del Hospital de Ninos J.M. de los Rios, Caracas, Venezuela. El sistema se encuentra desplegado en produccion en <https://obrasjm.space-z.ai/> con integracion continua via Vercel.

La auditoria abarca la arquitectura del sistema, el esquema de base de datos, las politicas de seguridad (RLS), los endpoints API, los componentes frontend, el flujo de autenticacion y autorizacion, y la infraestructura de despliegue. Se identifican hallazgos criticos, deuda tecnica y oportunidades de mejora.

Indicador	Valor
Framework	Next.js 16 (App Router)
Lenguaje	TypeScript 5
Base de Datos	Supabase (PostgreSQL + RLS)
Roles de Usuario	6 roles diferenciados
Endpoints API	20+ rutas REST
Tablas Principales	8 tablas con RLS habilitado
Despliegue	Vercel (CI/CD desde GitHub)
Componentes UI	50+ componentes shadcn/ui

Tabla 1. Indicadores generales del sistema

2. Arquitectura del Sistema

2.1 Vision General

ObrasJM es una aplicacion web monolitica construida con Next.js 16 App Router que sigue el patron de renderizado del lado del servidor (SSR) con hidratacion del cliente. La aplicacion utiliza un modelo de route unico (SPA-like) donde toda la interfaz se renderiza dentro de / (src/app/page.tsx) utilizando un sistema de tabs para la navegacion interna.

La arquitectura sigue un patron de tres capas: Frontend (React + shadcn/ui), Backend API (Next.js API Routes) y Base de Datos (Supabase PostgreSQL con RLS).

Capa	Tecnologia	Funcion
Presentacion	React 19 + Tailwind CSS 4 + shadcn/ui	UI interactiva, formularios, tablas, graficos
API	Next.js 16 API Routes	Logica de negocio, validacion, orquestacion

Datos	Supabase (PostgreSQL)	Persistencia, RLS, triggers, vistas
Autenticacion	Supabase Auth	JWT, sesiones, gestion de contraseñas
Almacenamiento	Supabase Storage	Fotos de evidencia (bucket publico)

Tabla 2. Capas de la arquitectura

2.2 Flujo de Datos

El flujo de datos sigue el patron clasico de aplicaciones Supabase: el cliente React realiza peticiones HTTP a los API Routes de Next.js, quienes a su vez utilizan el SDK de Supabase (`@supabase/ssr`) para interactuar con la base de datos. Las politicas RLS aplican automaticamente a nivel de base de datos, garantizando que cada usuario solo acceda a los datos permitidos para su rol.

El dashboard utiliza un cliente admin (service role key) para calcular metricas PAF agregadas sin restricciones RLS, mientras que las operaciones CRUD de alcance y avance utilizan el cliente autenticado estandar sujeto a RLS.

2.3 Single Page Architecture

La aplicacion implementa un patron SPA dentro del framework Next.js App Router: existe una unica ruta (/) y toda la navegacion se gestiona mediante tabs controlados por estado React. Las secciones principales son:

- Dashboard: Vista de KPIs globales, graficos PAF y heatmap del hospital
- Alcance Planificado: CRUD de items de alcance con filtros y paginacion
- Avance Ejecutado: CRUD de reportes de avance con aprobacion y fotos
- Administracion: Gestion de usuarios, unidades, especialidades, sectores (solo admin)

3. Stack Tecnologico

Categoria	Tecnologia	Version
Framework	Next.js	16.x
Lenguaje	TypeScript	5.x
Estilos	Tailwind CSS	4.x
Componentes UI	shadcn/ui (Radix)	Latest
Base de datos	Supabase (PostgreSQL)	SaaS
ORM (local)	Prisma	6.x
Graficos	Recharts	2.x

Fechas	date-fns	4.x
Iconos	Lucide React	0.525+
Animaciones	Framer Motion	12.x
Notificaciones	Sonner	2.x
Formularios	React Hook Form + Zod	7.x / 4.x

Tabla 3. Stack tecnologico detallado

4. Esquema de Base de Datos

4.1 Tablas Principales

El esquema consta de 8 tablas principales con tipos ENUM personalizados para garantizar la integridad de datos a nivel de base de datos. Las tablas utilizan UUID como claves primarias y timestamps con zona horaria (TIMESTAMPTZ).

Tabla	Descripcion	Campos Clave	RLS
profiles	Perfiles de usuario, extiende auth.users	rol, unidad_ejecutora_id, activo, debe_cambiar_password	Si
unidades_ejecutoras	Empresas contratistas	nombre, rif, contacto, activa	Si
especialidades	Tipos de trabajo (16 especialidades)	nombre (UNIQUE)	Si
sectores	Torres del hospital (6 sectores)	nombre, codigo (UNIQUE)	Si
subsectores	Pisos/areas dentro de sectores (27+)	sector_id (FK), nombre, codigo	Si
alcance_planificado	Items de trabajo planificado	peso_relativo, cantidad_planificada, status	Si
avance_ejecutado	Reportes de avance con evidencia	cantidad_reportada, status_aprobacion, inspector_id	Si

Tabla 4. Esquema de base de datos

4.2 Tipos ENUM

Tipo ENUM	Valores	Uso
-----------	---------	-----

user_rol	administrador, contratista, inspector, ingeniera_residente, directivo_hospital, ingenieria_hospital	profiles.rol
trabajo_tipo	Planificado, Imprevisto	avance_ejecutado.tipo_trabajo
aprobacion_status	Pendiente, Aprobado, Rechazado	avance_ejecutado.status_aprobacion
alcance_status	Activo, Completado, Suspendido	alcance_planificado.status

Tabla 5. Tipos ENUM del sistema

4.3 Vistas Calculadas

Se definen tres vistas SQL para el calculo del PAF que consolidan la informacion de alcances y avances aprobados:

- **v_paf_subsector:** Calcula el porcentaje de avance por cada item de alcance, sumando cantidades ejecutadas aprobadas vs. planificadas. Cap a 100%.
- **v_paf_sector:** Promedio ponderado del PAF por sector, considerando pesos relativos.
- **v_paf_global:** PAF global del proyecto con totales de items y items con avance.

Nota: La vista v_paf_subsector solo considera avances con status_aprobacion = Aprobado para el calculo, garantizando que solo trabajo verificado contribuya al PAF.

4.4 Triggers y Funciones

- **update_updated_at():** Trigger BEFORE UPDATE en profiles, alcance_planificado y avance_ejecutado para mantener updated_at automaticamente.
- **handle_new_user():** Trigger AFTER INSERT en auth.users que crea automaticamente un perfil en la tabla profiles con los datos del user_metadata del registro. Funciona como SECURITY DEFINER para sortear RLS en la insercion inicial.

Hallazgo: La funcion handle_new_user() no incluye los campos telefono y ente_pertenece en el INSERT original del schema.sql base, pero fueron anadidos posteriormente en la migracion migration-new-roles.sql. Existe inconsistencia entre los archivos SQL de referencia.

5. Autenticacion y Autorizacion

5.1 Flujo de Autenticacion

La autenticacion se gestiona mediante Supabase Auth con sesiones basadas en cookies JWT. El flujo es el siguiente:

- El usuario ingresa credenciales en el formulario de login (LoginPage)
- POST /api/auth/login valida contra Supabase Auth con signInWithPassword

- Supabase establece cookies HTTP-only para mantener la sesion
- El middleware de Next.js (src/middleware.ts) ejecuta updateSession en cada peticion para refrescar el token
- El contexto React (AuthProvider) sincroniza el estado de autenticacion via onAuthStateChange

Se implementa un guardia de autenticacion (AuthGuard) que muestra la LandingPage para usuarios no autenticados y fuerza el cambio de contrasena cuando debe_cambiar_password = true.

5.2 Sistema de Roles

El sistema define 6 roles con permisos diferenciados tanto a nivel de API como de RLS:

Rol	Dashboard	Alcance	Avance	Admin
administrador	Si	CRUD + Delete	CRUD + Aprobacion	Completo
contratista	Si	Solo lectura	Crear + Editar	No
inspector	Si	Crear + Editar	CRUD + Aprobacion	No
ingeniera_residente	Si	Solo lectura	Crear	No
directivo_hospital	Si	No acceso	Solo lectura	No
ingenieria_hospital	Si	Solo lectura	Solo lectura	No

Tabla 6. Matriz de permisos por rol

5.3 Politicas RLS (Row Level Security)

Las politicas RLS constituyen la capa fundamental de seguridad del sistema. Todas las tablas tienen RLS habilitado. A continuacion se resumen las politicas clave:

- **profiles:** Lectura para autenticados; INSERT/UPDATE solo admin; UPDATE propio perfil
- **alcance_planificado:** Lectura para autenticados; INSERT/UPDATE para admin e inspector; DELETE solo admin; lectura filtrada por unidad_ejecutora para contratistas e ingenieras residentes
- **avance_ejecutado:** Lectura para autenticados; INSERT para contratistas, residentes, inspectores y admin; UPDATE para inspectores y admin (todos los campos); DELETE y ALL para admin
- **especialidades/sectores/subsectores:** Lectura publica; gestion solo admin
- **unidades_ejecutoras:** Lectura para autenticados; gestion solo admin

Hallazgo de seguridad: Las politicas de UPDATE en avance_ejecutado permiten que un inspector modifique TODOS los campos de un avance (incluyendo cantidad_reportada, tipo_trabajo, fecha_reporte, fotos, notas), no solo el status_aprobacion. Si bien esto es consistente con la logica de negocio actual, podria considerarse una separacion de responsabilidades mas limpia si los inspectores solo pudieran modificar el status_aprobacion e inspector_id.

5.4 Gestion de Contraseñas

El sistema implementa un flujo robusto de gestión de contraseñas:

- Al crear un usuario, se establece `debe_cambiar_password = true`
- En el primer inicio de sesión, se fuerza el cambio de contraseña
- El cambio requiere contraseña actual, nueva contraseña (min. 6 caracteres) y confirmación
- El administrador puede resetear la contraseña de cualquier usuario, generando una temporal
- La contraseña temporal se puede copiar al portapapeles desde el diálogo de reset
- `POST /api/admin/users/[id]/reset-password` utiliza el service role key para actualizar la contraseña en `auth.users`

6. Endpoints API

6.1 Catálogo de Rutas

Metodo	Ruta	Descripción	Auth
POST	/api/auth/login	Inicio de sesión	No
POST	/api/auth/logout	Cerrar sesión	No
GET	/api/auth/me	Obtener perfil actual	Si
POST	/api/auth/register	Registro de usuario (admin)	Si
POST	/api/auth/change-password	Cambiar contraseña propia	Si
GET	/api/dashboard	Datos KPI para dashboard	Service
GET/POST	/api/alcance	Listar/crear alcances	Si
GET/PUT/DELETE	/api/alcance/[id]	CRUD alcance individual	Si
GET/POST	/api/avance	Listar/crear avances	Si
GET/PUT	/api/avance/[id]	Obtener/editar avance	Si
GET/POST	/api/admin/users	Listar/crear usuarios	Admin
PUT/DELETE	/api/admin/users/[id]	Editar/eliminar usuario	Admin
POST	/api/admin/users/[id]/reset-password	Resetear contraseña	Admin
CRUD	/api/especialidades	Gestión de especialidades	Si

CRUD	/api/sectores	Gestion de sectores	Si
CRUD	/api/subsectores	Gestion de subsectores	Si
CRUD	/api/unidades-ejecutoras	Gestion de unidades	Si

Tabla 7. Catalogo de endpoints API

6.2 Patrones de Seguridad API

Todos los endpoints protegidos siguen un patron consistente de verificacion:

- 1. Verificar autenticacion via supabase.auth.getUser()
- 2. Obtener perfil del usuario desde la tabla profiles
- 3. Verificar permisos basados en el rol (profile.rol)
- 4. Ejecutar la operacion con el cliente Supabase autenticado (sujeto a RLS)
- 5. Para operaciones administrativas, usar createAdminClient() con service role key

Hallazgo: El endpoint GET /api/dashboard utiliza el admin client (service role key) para calcular metricas sin restricciones RLS. Esto es apropiado para datos agregados, pero el endpoint no verifica roles, permitiendo que cualquier usuario autenticado acceda a las metricas globales. Esto es consistente con el diseno actual donde todos los roles ven el mismo dashboard.

7. Componentes Frontend

7.1 Estructura de Componentes

Componente	Archivo	Lineas	Descripcion
AppContent	page.tsx	~410	Layout principal con header, tabs, footer
DashboardView	dashboard/dashboard-view.tsx	~124	Dashboard con KPIs, heatmap y graficos
AlcanceView	alcance/alcance-view.tsx	~805	CRUD de alcances con filtros y paginacion
AvanceView	avance/avance-view.tsx	~1100+	CRUD de avances, aprobacion, galeria fotos
AdminView	admin/admin-view.tsx	~1500+	Gestion usuarios, unidades, especialidades, sectores
AuthGuard	auth-guard.tsx	~53	Guardia de autenticacion y cambio de contrasena
LandingPage	landing/landing-page.tsx	N/A	Pagina publica con login y galeria

Tabla 8. Componentes frontend principales

7.2 Patrones de Diseno UI

- **Responsive Design:** Uso extensivo de breakpoints de Tailwind (sm:, md:, lg:) para adaptar la interfaz a movil y desktop
- **Shadcn/ui:** 50+ componentes de la libreria shadcn/ui (Dialog, Table, Select, Badge, etc.)
- **Formularios modales:** Todas las operaciones CRUD utilizan Dialog components con validacion
- **Paginacion:** Paginacion cliente-side con ITEMS_PER_PAGE = 10 y navegacion con ellipsis
- **Filtros:** Selectores de filtro por sector, especialidad, status y rango de fechas
- **Upload de archivos:** Sistema de subida de fotos con preview, validacion de tipo y tamano (10MB max)
- **Calendario:** Date picker con Calendar component de shadcn/ui para seleccion de fechas
- **Feedback:** Notificaciones toast via Sonner para exito, error y advertencia
- **Loaders:** Skeleton loading states para tablas y spinner para envios
- **Sticky footer:** Layout con min-h-screen flex flex-col y mt-auto en footer

7.3 Dashboard y Visualizacion

El dashboard presenta tres secciones principales:

- **KPI Cards:** Tarjetas con PAF Global (%), Frentes Activos y Alertas (avances pendientes)
- **Hospital Heatmap:** Mapa de calor visual por sector mostrando el nivel de avance
- **PAF Chart:** Grafico de barras por sector usando Recharts

El dashboard se actualiza automaticamente cada 30 segundos y cuenta con un boton de actualizacion manual. Utiliza el API endpoint /api/dashboard que calcula todas las metricas en el servidor usando el admin client de Supabase.

8. Evaluacion de Seguridad

8.1 Fortalezas

- RLS habilitado en todas las tablas - defensa en profundidad
- Validacion de roles tanto en API como en RLS - doble verificacion
- Service role key solo se usa en endpoints administrativos
- Contraseñas con requisito de longitud minima (6 caracteres)
- Force change password en primer inicio de sesion
- Usuarios inactivos no pueden iniciar sesion (verificacion en login)
- Middleware de sesion que refresca tokens automaticamente
- Cookies HTTP-only para sesiones (manejadas por Supabase SSR)

8.2 Areas de Mejora

Area	Riesgo	Severidad	Descripcion
Validacion input	Inyeccion/SQLi	Media	No se usa Zod en endpoints API; solo validacion manual de campos requeridos
Rate limiting	Brute force	Alta	No hay rate limiting en endpoints de autenticacion
HTTPS enforcement	Intercepcion	Baja	Depende de la configuracion de Vercel/Supabase
Audit log	Trazabilidad	Media	No existe registro de auditoria para acciones criticas
CSRF	Falsificacion	Baja	Protegido implicitamente por cookies SameSite de Supabase
Password policy	Credenciales debiles	Media	Solo longitud minima de 6; sin requisitos de complejidad
Session timeout	Sesion abierta	Media	Sin timeout inactivo configurado; sesion persiste hasta cierre manual

Tabla 9. Evaluacion de seguridad - areas de mejora

9. Deuda Tecnica y Oportunidades de Mejora

9.1 Deuda Tecnica Identificada

Item	Prioridad	Descripcion
Componentes monoliticos	Alta	AvanceView (~1100 lineas) y AdminView (~1500+ lineas) son componentes excesivamente grandes. Deberian dividirse en subcomponentes.
Type casting inseguro	Alta	Uso extensivo de "as Record" en enriquecimiento de datos API. Se pierde type safety en las uniones de tablas.
Schema SQL fragmentado	Media	Existen 7 archivos SQL (step1-step6 + schema + 3 migraciones) con inconsistencias entre ellos. No hay un sistema de migraciones formal.
Paginacion cliente-side	Media	Todos los datos se cargan en el cliente y se pagan en memoria. Con datos grandes, deberia ser server-side.
Sin manejo de errores centralizado	Media	Cada endpoint maneja errores individualmente. Falta un middleware de error handling y logging consistente.
next.config.ts	Baja	ignoreBuildErrors: true y reactStrictMode: false en produccion. Deberian eliminarse una vez estabilizado el proyecto.
Prisma no utilizado	Baja	Prisma esta instalado y configurado pero no se usa. El ORM real es Supabase Client. Prisma genera confusion y peso innecesario.

Upload sin endpoint visible	Media	El frontend llama a POST /api/upload pero no se encontro el archivo de ruta. Posiblemente eliminado o pendiente de implementacion.
-----------------------------	-------	--

Tabla 10. Deuda tecnica identificada

9.2 Mejoras Funcionales Sugeridas

- **Exportar datos:** Permitir exportacion de alcances y avances a Excel/PDF para reportes
- **Notificaciones:** Sistema de notificaciones para alertar sobre avances pendientes de aprobacion
- **Historial de cambios:** Audit log para rastrear modificaciones en alcances y avances
- **Filtros avanzados:** Filtrar avances por inspector, especialidad o unidad ejecutora
- **Dashboard por rol:** Vista diferenciada del dashboard segun el rol del usuario
- **Grafico de tendencias:** Evolucion del PAF en el tiempo (line chart temporal)
- **Modo offline:** Soporte PWA para reportar avances sin conexion
- **Adjuntar documentos:** Permitir subir PDFs ademas de fotos como evidencia
- **Comentarios en aprobacion:** Campo de notas obligatorio al rechazar un avance
- **Paginacion server-side:** Mover la paginacion al backend para escalabilidad

10. Infraestructura y Despliegue

10.1 Configuracion de Despliegue

- **Plataforma:** Vercel (hosting serverless)
- **Dominio:** obrasjm.space-z.ai
- **CI/CD:** Despliegue automatico desde GitHub (jm-obras/ObrasJM)
- **Base de datos:** Supabase Cloud (PostgreSQL gestionado)
- **Storage:** Supabase Storage (bucket "evidencias" publico)
- **SSL:** Proporcionado por Vercel automaticamente

El flujo de despliegue es: commit a la rama principal de GitHub -> Vercel detecta el cambio -> build automatico -> deploy en produccion. No existen ambientes de staging o preview configurados.

10.2 Variables de Entorno

El sistema requiere las siguientes variables de entorno para funcionar:

- NEXT_PUBLIC_SUPABASE_URL: URL del proyecto Supabase
- NEXT_PUBLIC_SUPABASE_ANON_KEY: Clave anonima (publica)
- SUPABASE_SERVICE_ROLE_KEY: Clave de servicio (secreta, solo servidor)

Hallazgo: La clave SUPABASE_SERVICE_ROLE_KEY es accesible en todos los API routes del servidor. Es critico garantizar que nunca se exponga al cliente (las variables sin prefijo NEXT_PUBLIC_ son solo de servidor en Next.js).

11. Migraciones y Evolucion del Schema

El esquema de base de datos ha evolucionado a traves de multiples migraciones aplicadas manualmente en el SQL Editor de Supabase:

Archivo	Descripcion
schema.sql	Schema completo inicial (tablas + datos seed + RLS + triggers + vistas)
step1-tables.sql	Paso 1: Creacion de enums y tablas
step2-seed-data.sql	Paso 2: Datos iniciales (especialidades, sectores, subsectores)
step3-rls-policies.sql	Paso 3: Habilitar RLS y crear politicas
step4-functions-triggers.sql	Paso 4: Funciones y triggers
step5-views.sql	Paso 5: Vistas de calculo PAF
step6-storage.sql	Paso 6: Configuracion de Storage bucket
migration-new-roles.sql	Migracion: Nuevos roles + campos telefono, ente_pertenece, debe_cambiar_password
migration-inspector-alcance-policies.sql	Migracion: Permisos de inspector para alcances
migration-inspector-avance-policies.sql	Migracion: Permisos de inspector para avances

Tabla 11. Archivos de migracion SQL

Recomendacion: Implementar un sistema formal de migraciones (Supabase CLI o herramienta similar) para rastrear versiones del schema y evitar inconsistencias. Los archivos step1-6 contienen el schema base con las migraciones ya integradas, lo que genera confusion sobre cual archivo ejecutar.

12. Conclusiones y Recomendaciones

12.1 Estado Actual

ObrasJM es un sistema funcional que cumple su objetivo principal de control y seguimiento del PAF del Hospital de Ninos J.M. de los Rios. La arquitectura basada en Next.js + Supabase proporciona una base solida con RLS como capa de seguridad fundamental. El sistema maneja correctamente los 6 roles de usuario con permisos diferenciados y ofrece una interfaz responsiva y moderna.

12.2 Recomendaciones Prioritarias

Alta prioridad:

- Implementar rate limiting en endpoints de autenticacion
- Refactorizar componentes monoliticos (AvanceView, AdminView) en subcomponentes
- Agregar validacion de esquema con Zod en todos los endpoints API
- Verificar y documentar el endpoint de upload de archivos

Media prioridad:

- Implementar sistema de migraciones formal
- Agregar audit logging para acciones criticas
- Mover paginacion a server-side para escalabilidad
- Mejorar la politica de contraseñas (complejidad, expiracion)
- Implementar session timeout por inactividad

Baja prioridad:

- Eliminar Prisma si no se utiliza (reduccion de dependencias)
- Corregir next.config.ts (remove ignoreBuildErrors)
- Agregar tests automatizados (unit + integration)
- Consolidar archivos SQL en un sistema de migraciones versionado

12.3 Metricas deCodigo

Metrica	Valor
Archivos TypeScript (src/)	~30+
Archivos SQL (supabase/)	11
Endpoints API	20+
Componentes UI (shadcn/ui)	50+
Componentes custom	~15
Dependencias (package.json)	60+
Lineas de codigo estimadas	~5,000+
Tipos TypeScript definidos	12 interfaces/tipos

Tabla 12. Metricas de codigo

